

Process Management

2026 Semester 2 COMPSCI 340: Operating Systems

Talia Xu

Lecture 6

2.1.0

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

Process Management Continues After fork and exec

The previous lecture created processes and selected the programs they run.

This lecture follows those processes through the rest of their lifetime:

- Parent and child processes form a process tree
- A parent can use `wait` to collect a child's result
- Different exit orders create zombie or orphan processes

Unix Processes Form a Parent-Child Tree

- fork creates a child and records which process created it
- The child has its own PID and records its parent's PID
- A parent may have many children; a child has one current parent
- The relationship remains important after either process exits

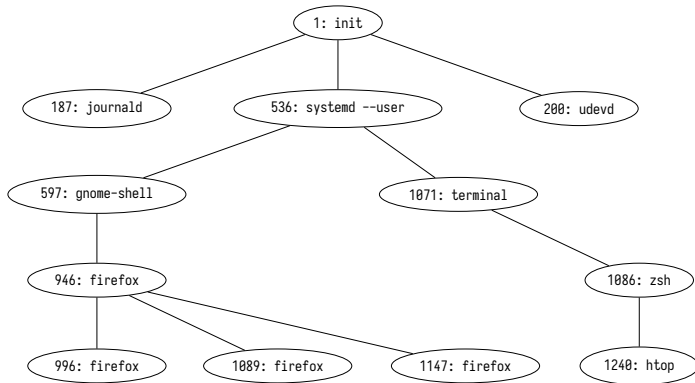
The process tree gives the operating system a clear owner for each child's exit result.

PID 1 Starts the User-Space Process Tree

After the kernel initializes, it starts the first user-space process with process ID 1.

- Traditional Unix calls this role `init`
- Many Linux systems run `systemd` as PID 1
- PID 1 starts services that create more processes
- It also helps collect processes that lose their original parent

A Process Tree Shows Who Created Whom



PIDs Identify Active Processes

```
pid_t me = getpid();  
pid_t parent = getppid();
```

- A PID is unique among active processes in one PID namespace
- The kernel may reuse a PID after the old process is collected
- `getppid()` reports the process's current parent

Termination Leaves a Result for the Parent

When a process exits:

1. The kernel closes its resources and records how it terminated
2. The parent is notified that a child changed state
3. A small process record remains until the result is collected
4. The parent calls `wait` or `waitpid` to collect it

The exited process no longer executes, but its PID and termination information cannot be discarded yet.

wait Collects a Child's Result

```
pid_t wait(int *wstatus);  
pid_t waitpid(pid_t pid, int *wstatus, int options);
```

Situation	What the default wait does
Exited child	Returns that child's PID immediately
Running children	Blocks until one exits
No children	Returns -1 with errno = ECHILD

If wstatus is not NULL, macros such as WIFEXITED and WEXITSTATUS interpret the stored result.

Trace the Order Created by wait

```
int main(void) {  
    pid_t pid = fork();  
  
    if (pid == 0) {  
        printf("child\n");  
        return 0;  
    }  
  
    wait(NULL);  
    printf("parent\n");  
    return 0;  
}
```

wait Makes the Parent Print After the Child

```
child  
parent
```

1. The parent may reach `wait` before or after the child runs
2. `wait` cannot return until the child exits
3. The parent therefore prints only after the child's result is available

`wait` constrains the order without choosing which process runs first.

If the Child Exits First, It Becomes a Zombie

A **zombie** is a child that has terminated but has not yet been collected by its parent.

- It is not running and keeps no user address space
- The kernel retains its PID, exit status, and accounting information
- The record disappears when the parent performs a successful wait

A program that continually creates children but never waits can eventually exhaust process-table entries.

Observe a Child Before and After It Exits

```
pid_t child = fork();
```

```
if (child == 0) {  
    sleep(2);  
    return 0;  
}
```

```
sleep(1);  
print_state(child);  
sleep(2);  
print_state(child);  
waitpid(child, NULL, 0);
```

print_state reads the child's state from /proc.

The Child Changes from Sleeping to Zombie

Point	Child state	Reason
First check	Sleeping	The child is inside sleep
Child exits	Zombie	Its result is waiting for the parent
Parent waits	Removed	The result has been collected

The transition to zombie happens at exit; waitpid removes the remaining process record.

If the Parent Exits First, the Child Is Reparented

A living child whose parent exits is called an **orphan**.

- The child continues running
- The kernel assigns it to a process that can later collect its result
- This is often PID 1, but a nearer process may act as a subreaper
- `getppid()` reports the child's new parent

Reparenting preserves the rule that every exiting child has a process responsible for collecting it.

Observe a Child After Its Parent Exits

```
pid_t pid = fork();

if (pid == 0) {
    printf("parent: %d\n", getppid());
    sleep(2);
    printf("parent after sleep: %d\n", getppid());
}
else {
    sleep(1);
}
```

getppid Shows the Child's New Parent

The first line reports the process that called fork.

While the child sleeps, the original parent exits. The kernel reparents the child, so the second line reports a different PID.

The replacement is commonly PID 1 or a subreaper. The exact number depends on the environment, especially inside a container.

Parent-Child Cleanup Has Two Cases

Exit order	Temporary situation	Resolution
Child exits first	Zombie child	Original parent calls wait
Parent exits first	Orphan child	Child is assigned to a reaper

- A zombie has finished but still has a result to collect
- An orphan is still running but now has a different parent

wait Blocks Only When No Result Is Ready

Case	Result	Reason
1	Returns -1	There is no child to wait for
2	Blocks	The child has not exited
3	Returns immediately	The zombie is ready to collect
4	Returns immediately	It collects the zombie; the other child continues

Waiting is about an available child result, not merely whether a child process exists.

Compare Four Calls to wait

Parent process P calls the default wait in each situation:

1. P has no children
2. P has one child that is still running
3. P has one child that is a zombie
4. P has two children: one running and one zombie

Trace a Child Chain Without wait

```
int i = 4;

while (i != 0) {
    pid_t pid = fork();

    if (pid == 0)
        i--;
    else {
        printf("%d\n", i);
        exit(0);
    }
}
```

Trace the Same Child Chain with waitpid

```
int i = 4;

while (i != 0) {
    pid_t pid = fork();

    if (pid == 0)
        i--;
    else {
        waitpid(pid, NULL, 0);
        printf("%d\n", i);
        exit(0);
    }
}
```

Summary: Parents Manage How Children Finish

- Unix processes form a parent-child tree rooted at PID 1
- PIDs identify active processes and may be reused after cleanup
- Exiting leaves a result that a parent collects with `wait`
- A child that exits first is a zombie until its parent collects it
- A child whose parent exits first is reparented to a process that can collect it later
- `wait` also provides ordering between parent and child execution

Key distinction: a zombie has finished but still has a parent; an orphan is still running but has received a new parent.